PROMPT FOR NGUMUS EYE VERCEL DEVELOPER

Request: Complete API Endpoint Integration Documentation for Replit Mobile Developer

---

CONTEXT & BACKGROUND

I am building the **Ngumus Eye** mobile app on Replit (React Native / Expo) to push to the App Store and Google Play. The backend/web app is built on Vercel (Next.js). I need you to produce a **complete, exhaustive API endpoint integration documentation** — similar to a developer handoff document — so the Replit mobile developer can connect the mobile app to our Vercel backend database and logic via API endpoints.

This document must cover **every single screen, every button, every action, every form submission, every data fetch, every status check** — nothing can be omitted. The mobile developer will use this document as their sole reference, so it must be 100% complete.

WHAT I NEED YOU TO PRODUCE

Please create a **Markdown document** titled:

> **"Ngumus Eye Mobile App — Complete API Documentation for Replit Developer"**

The document must follow this exact format and level of detail for EVERY action in the app:

REQUIRED FORMAT FOR EVERY ENDPOINT / ACTION

For each screen, button, or action, document the following:

[Section Number]. [Screen/Action Name]

Page: [Screen name in the app]
Trigger: [What triggers this — e.g., button press, page load, form submit]
API Endpoint: [METHOD /api/path]

User Input (if any):
- Field name (input type, validation rules)

Request Body:
```json
{ exact JSON structure }
```

Success Response ([status code]):
```json
{ exact JSON structure }
```
- What to do after success (navigate to X, store Y in state, etc.)

Error Responses:
- [status code]: [meaning]
```json
{ error JSON }
```

UI Requirements:
- Disable buttons while loading
- Show error messages where
- Any conditional logic (show/hide elements based on response)
- Navigation rules

COVERAGE REQUIREMENTS — INCLUDE ALL OF THE FOLLOWING

Please cover **every one of these areas** in the Ngumus Eye app. For each area, document every screen, every button, every API call, every data load, and every user action:

1. AUTHENTICATION & SESSION MANAGEMENT

- Login screen (every input field, submission, error states)
- Session handling (how session/token is stored and sent)

- Logout (button action, what gets cleared, where user is redirected)
- Session expiry handling (what happens when token expires mid-use)
- Any biometric or PIN authentication if applicable

2. REGISTRATION / ONBOARDING FLOW (EVERY STEP)

- Every step of the registration process in order
- Every form field on every step
- Uniqueness checks (phone, email, ID, username — whatever applies)
- OTP sending and verification (if applicable)
- Profile photo / avatar upload (if applicable)
- Any document or KYC uploads (if applicable)
- Account creation final submission (full request body with every field)
- Post-registration navigation

3. FORGOT PASSWORD / RESET FLOW

- Every step (request reset, OTP/link verification, set new password)
- Full request/response for each step

4. HOME / DASHBOARD SCREEN

- Every piece of data loaded on page entry (all API calls)
- Every widget, card, stat, or metric displayed (where data comes from)
- Any quick-action buttons and what they trigger
- Notifications or alerts loaded on dashboard

5. USER PROFILE & SETTINGS

- Load profile data (full response structure — every field)
- Edit profile (every editable field, submission endpoint)
- Change password / PIN
- Notification preferences
- Any account settings toggles
- Delete account (if applicable)

6. [NGUMUS EYE CORE FEATURE SCREENS — INSERT ALL FEATURE-SPECIFIC SCREENS]

> **IMPORTANT:** List and document EVERY major feature screen of Ngumus Eye here. For each screen include:

- All data loaded on entry (GET requests, Supabase queries, or both)
- All actions the user can take (buttons, forms, toggles, sliders, uploads)
- Every API call those actions trigger
- Full request and response structures
- Error states and how to handle them
- UI conditional logic (what shows/hides based on data or status)

7. IMAGE / VIDEO / FILE UPLOAD FLOWS (IF APPLICABLE)

- Upload endpoint(s) — method, content type, field names
- File size and type restrictions
- Success and error handling
- Where uploaded file URL/reference is stored

8. SEARCH & FILTERING (IF APPLICABLE)

- Search input behavior (on-type vs on-submit)
- API endpoint, query parameters
- Results rendering
- Empty state handling

9. NOTIFICATIONS

- How notifications are fetched (polling or push)
- Endpoint and response structure
- Mark as read / mark all as read actions
- Notification count badge update logic

10. PAYMENT / TRANSACTION FLOWS (IF APPLICABLE)

- Every step of any payment or transaction flow
- All form fields and validation
- Confirmation screens and what data they display
- Submission endpoint — full request body

- Success / failure / pending handling
- Transaction history screen — data structure, pagination

11. LISTINGS, FEEDS, OR DATA TABLES

- Every list or feed screen
- Pagination or infinite scroll logic (page size, cursor, offset)
- Pull-to-refresh behavior
- Sort/filter controls and their API parameters
- Detail view navigation (what data is passed vs re-fetched)

12. DETAIL / SINGLE-ITEM SCREENS

- What data is fetched on entry (full response structure)
- All actions available (buttons, menus)
- Status fields and their possible values with display labels and colors
- Delete / archive / status change actions

13. ADMIN OR MANAGEMENT SCREENS (IF APPLICABLE)

- Every admin-only action
- Approval / rejection flows
- Status update endpoints

14. DIRECT SUPABASE QUERIES (IF ANY)

- Any screens that query Supabase directly instead of via API route
- Exact table names and query logic
- RLS policy considerations the mobile dev must know

15. REAL-TIME / SUBSCRIPTIONS (IF APPLICABLE)

- Any real-time data (Supabase subscriptions, websockets)
- What tables or channels to subscribe to
- How to update UI on real-time events

STATUS CODES & UTILITIES SECTION

At the end of the document, include:

1. **All custom status/state codes used in the app** — with label and display color (e.g., application statuses, loan statuses, user statuses, listing statuses, etc.)
2. **Phone number formatting** — if phone numbers are used, document expected format
3. **Date formatting** — timezone, display format
4. **Currency formatting** — if monetary values are used
5. **Common HTTP status codes and what mobile app should do for each**
6. **Standard error response format**
7. **All environment variables the mobile app needs** (public keys only — no secrets)

ERROR HANDLING SECTION

Document:

- Standard error response JSON structure
- Every known API error code or message
- What the mobile app should show the user for each
- Network error handling
- Authentication error handling (redirect to login, etc.)

TESTING CHECKLIST SECTION

Produce a checkbox-style testing checklist covering every major flow so the Replit developer can verify their integration is working correctly before submission to app stores.

FINAL NOTES SECTION

Include:

- What the mobile app **MUST NOT** do (calculations, direct DB writes, etc.)
- What the mobile app **MUST** do (fetch, display, validate input, handle errors)
- Clear **Backend vs Frontend responsibility split**

- Base URL for API
- Any required headers (auth headers, content-type, etc.)
- Cookie vs token-based auth — how session is maintained in mobile context

---

IMPORTANT INSTRUCTIONS FOR YOU (VERCEL DEV)

1. **Do not omit anything.** Every button, every form, every page load, every status check — document it. If a screen makes 3 API calls on load, document all 3.
2. **Include the exact JSON.** Not pseudocode — real field names as they exist in the database and API. The Replit developer cannot guess field names.
3. **Include every field in every request and response.** If a response returns 20 fields, show all 20. The mobile developer needs to know what's available.
4. **Document conditional UI logic.** If a button only shows when `status === "approved"`, say that. If a field only appears for a certain user type, document it.
5. **Document navigation.** After every successful API call, where does the user go next?
6. **Use the exact same format throughout.** Consistent structure makes it easy for the mobile developer to follow.
7. **Flag anything that uses Supabase directly** vs going through a Next.js API route. Both are acceptable, but the mobile developer must know which approach to use and why.
8. **Include all multipart/form-data endpoints** with field names and file constraints.
9. **Document any rate limits** or throttling the mobile app needs to respect.
10. **The mobile app is the Replit version of the Ngumus Eye Vercel app.** The mobile developer needs to replicate every user-facing function. Think of every flow a user can go through and document the API backing it.

OUTPUT FORMAT

- Format: Markdown (.md)
- Version: Include version number and date
- Table of contents with anchor links
- Every section numbered
- All JSON examples in fenced code blocks with `json` syntax highlighting
- All endpoint paths bold or in inline code

Deliver this as a single, complete Markdown document. Do not split it into multiple files. Do not leave any section as "to be completed later." This document must be production-ready and complete.

This documentation will be used by the Replit mobile developer to build and submit the Ngumus Eye mobile app to the Apple App Store and Google Play Store. Accuracy and completeness are critical.